

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, FunctionTransformer
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.cluster import KMeans
```



```
# Step 1 - Load data
DATA_PATH = "/content/housing.csv"

if os.path.exists(DATA_PATH):
    df = pd.read_csv(DATA_PATH)
    print("Loaded uploaded dataset:", DATA_PATH)
else:
    from sklearn.datasets import fetch_california_housing
    data = fetch_california_housing(as_frame=True)
    df = data.frame.copy()
    df.columns = [c.replace('-', '_') for c in df.columns]
    print("Loaded sklearn california housing dataset")

print("Shape:", df.shape)
display(df.head())
print("\nColumns:", df.columns.tolist())
```

Loaded uploaded dataset: /content/housing.csv

Shape: (20640, 10)

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	populat
0	-122.23	37.88	41.0	880.0	129.0	32
1	-122.22	37.86	21.0	7099.0	1106.0	240
2	-122.24	37.85	52.0	1467.0	190.0	49
3	-122.25	37.85	52.0	1274.0	235.0	59
4	-122.25	37.85	52.0	1627.0	280.0	56

Columns: ['longitude', 'latitude', 'housing_median_age', 'total_rooms', 'total_b

```
# Step 2 - EDA
print("Info:")
display(df.info())

print("\nMissing values per column:")
print(df.isna().sum())

print("\nDescriptive statistics (numeric):")
display(df.describe().T)

# Quick histograms for numeric columns (first up to 8)
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
cols_to_plot = numeric_cols[:8]

plt.figure(figsize=(12,6))
for i, col in enumerate(cols_to_plot, 1):
    plt.subplot(2,4,i)
    plt.hist(df[col].dropna(), bins=30)
    plt.title(col)
plt.tight_layout()
plt.show()

# Boxplots to inspect outliers
plt.figure(figsize=(12,6))
for i, col in enumerate(cols_to_plot, 1):
    plt.subplot(2,4,i)
    plt.boxplot(df[col].dropna(), vert=False)
    plt.title(col)
plt.tight_layout()
plt.show()
```





```

Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   longitude              20640 non-null  float64
1   latitude               20640 non-null  float64
2   housing_median_age     20640 non-null  float64
3   total_rooms            20640 non-null  float64
4   total_bedrooms         20433 non-null  float64
5   population             20640 non-null  float64
6   households              20640 non-null  float64
7   median_income           20640 non-null  float64
8   median_house_value     20640 non-null  float64
9   ocean_proximity        20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
None

```

Missing values per column:

```

longitude      0
latitude       0
housing_median_age  0
total_rooms    0
total_bedrooms 207
population     0
households     0

```

```

# Step 3 - choose target
possible_targets = [c for c in df.columns if c.lower() in (
    'medhouseval', 'median_house_value', 'saleprice', 'price', 'house_value', 'ho

if possible_targets:
    target = possible_targets[0]
    print("Auto-detected target:", target)
else:
    # fallback: use last numeric column; change if incorrect
    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    target = numeric_cols[-1]
    print("No common target found. Using last numeric column as target:", ta

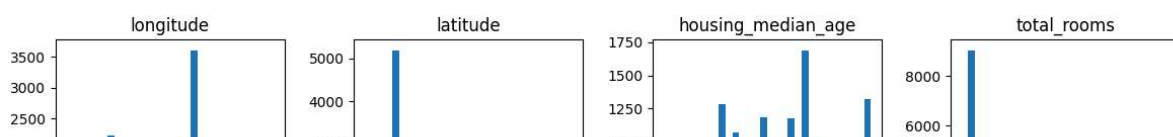
# Quick check
print("Target distribution summary:")
display(df[target].describe())

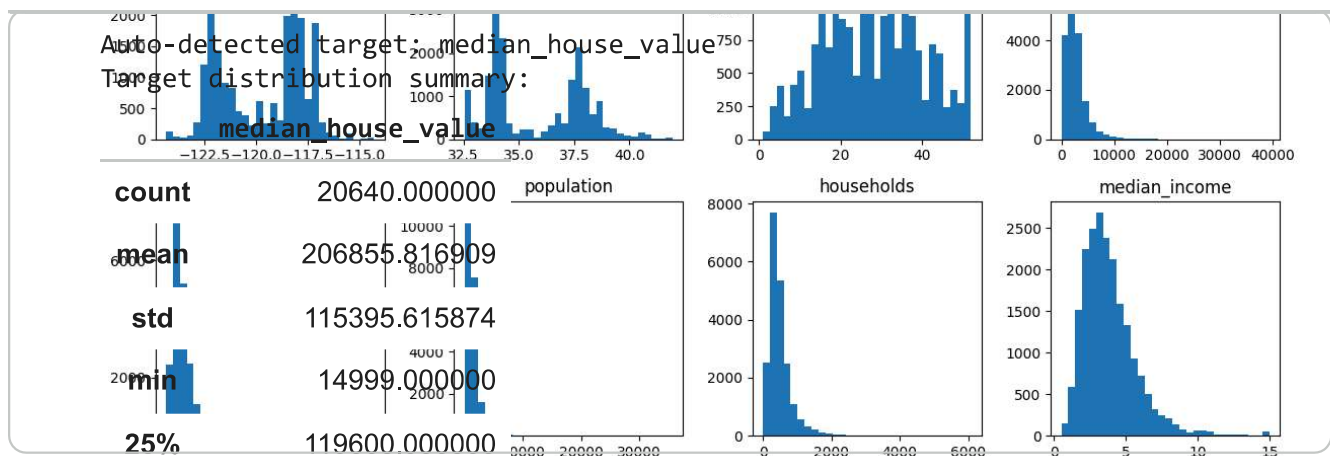
```

```

nousenoias      20640.0      499.539680      382.329753      1.0000      280.00
median_income    20640.0      3.870671      1.899822      0.4999      2.56
median_house_value 20640.0  206855.816909  115395.615874  14999.0000  119600.00

```





```
# Step 4 - create region if coordinates exist
if {'Latitude','Longitude'}.issubset(df.columns):
    coords = df[['Latitude','Longitude']].fillna(method='ffill').fillna(method='bfill')
    # choose clusters = 6 (tune if desired)
    kmeans = KMeans(n_clusters=6, random_state=42)
    df['region'] = kmeans.fit_predict(coords)
    print("Added 'region' column (kmeans clusters). Unique regions:", df['region'].nunique())
else:
    print("Latitude/Longitude not found - skipping 'region' creation.")
```

Latitude/Longitude not found - skipping 'region' creation.

```
# Step 5 - preprocessing pipelines
# Choose features
X = df.drop(columns=[target]).copy()
y = df[target].copy()

numeric_features = X.select_dtypes(include=[np.number]).columns.tolist()
if 'region' in numeric_features:
    numeric_features.remove('region')
categorical_features = [c for c in X.columns if X[c].dtype == 'object' or c == 'region']

print("Numeric features:", numeric_features[:10])
print("Categorical features (detected):", categorical_features[:10])

# IQR capping function
def cap_outliers_iqr(X_array):
    X = X_array.copy() # X is now a NumPy array
    # Apply capping to each column of the NumPy array
    for i in range(X.shape[1]):
        col_data = X[:, i]
        q1 = np.percentile(col_data, 25)
        q3 = np.percentile(col_data, 75)
        iqr = q3 - q1
        lower = q1 - 1.5 * iqr
        upper = q3 + 1.5 * iqr
        X[:, i] = np.clip(col_data, lower, upper)
```

```

return X

from sklearn.preprocessing import FunctionTransformer
cap_transformer = FunctionTransformer(cap_outliers_iqr)

numeric_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('cap', cap_transformer), # Apply capping after imputation
    ('scaler', StandardScaler())
])

categorical_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor = ColumnTransformer([
    ('num', numeric_pipeline, numeric_features),
    ('cat', categorical_pipeline, categorical_features)
], remainder='drop')

print("Preprocessor ready.")

```

```

Numeric features: ['longitude', 'latitude', 'housing_median_age', 'total_rooms',
Categorical features (detected): ['ocean_proximity']
Preprocessor ready.

```

```

# Step 6 - train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random
print("Train shape:", X_train.shape, "Test shape:", X_test.shape)

```

```

Train shape: (16512, 9) Test shape: (4128, 9)

```

```

near models
linear_model import LinearRegression, Ridge

Pipeline([('preproc', preprocessor), ('lr', LinearRegression())])
e = Pipeline([('preproc', preprocessor), ('ridge', Ridge(alpha=1.0))])

CV for speed
splits=3, shuffle=True, random_state=42)

e in [('LinearRegression', lr_pipeline), ('Ridge', ridge_pipeline)]:
cross_val_score(pipe, X_train, y_train, cv=kf, scoring='neg_mean_squared_errr
np.sqrt(-cv_mse).mean()
X_train, y_train)
ipe.predict(X_test)
= np.sqrt(mean_squared_error(y_test, preds))
r2_score(y_test, preds)

```

```

name} - CV RMSE: {cv_rmse:.4f} | Test RMSE: {test_rmse:.4f} | Test R²: {test_
icients for linear model with feature names
it(X_train, y_train)
ine.named_steps['preproc']
meric_features.copy()

named_transformers_['cat'].named_steps['onehot']
= list(ohe.get_feature_names_out(categorical_features))
ion:
= []
= num_cols + cat_names
ipeline.named_steps['lr'].coef_
pd.Series(coefs, index=feature_names).sort_values(key=abs, ascending=False)
linear coefficients (abs sorted):"
series.head(15))

```

```

LinearRegression - CV RMSE: 66484.7273 | Test RMSE: 70384.4061 | Test R²: 0.6226
Ridge - CV RMSE: 66477.1573 | Test RMSE: 70387.2146 | Test R²: 0.6219

```

Top linear coefficients (abs sorted):

	0
ocean_proximity_ISLAND	107667.039442
median_income	81433.708683
total_bedrooms	65829.355748
latitude	-52605.691346
longitude	-51669.780818
ocean_proximity_INLAND	-50991.434055
population	-50005.787083
total_rooms	-29127.134812
ocean_proximity_NEAR BAY	-25674.162067
households	17793.180724
housing_median_age	16453.789280
ocean_proximity_<1H OCEAN	-16325.145517
ocean_proximity_NEAR OCEAN	-14676.297803

dtype: float64

```

# Step 8 - Random Forest
rf_pipeline = Pipeline(['preproc', preprocessor),

```

```

('rf', RandomForestRegressor(n_estimators=100, random_s

cv_mse = cross_val_score(rf_pipeline, X_train, y_train, cv=kf, scoring='neg_me
cv_rmse = np.sqrt(-cv_mse).mean()
rf_pipeline.fit(X_train, y_train)
rf_preds = rf_pipeline.predict(X_test)
rf_test_rmse = np.sqrt(mean_squared_error(y_test, rf_preds))
rf_test_r2 = r2_score(y_test, rf_preds)
print(f"RandomForest - CV RMSE: {cv_rmse:.4f} | Test RMSE: {rf_test_rmse:.4f} |

# Feature importances
ct = rf_pipeline.named_steps['preproc']
num_cols = numeric_features.copy()
try:
    ohe = ct.named_transformers_['cat'].named_steps['onehot']
    cat_names = list(ohe.get_feature_names_out(categorical_features))
except Exception:
    cat_names = []
feature_names = num_cols + cat_names
rf_model = rf_pipeline.named_steps['rf']
if hasattr(rf_model, 'feature_importances_'):
    fi = pd.Series(rf_model.feature_importances_, index=feature_names).sort_val
    print("\nTop 15 feature importances (RandomForest):")
    display(fi.head(15))
    plt.figure(figsize=(8,4))
    fi.head(10).plot(kind='bar')
    plt.title("Top 10 RandomForest feature importances")
    plt.tight_layout()
    plt.show()

```

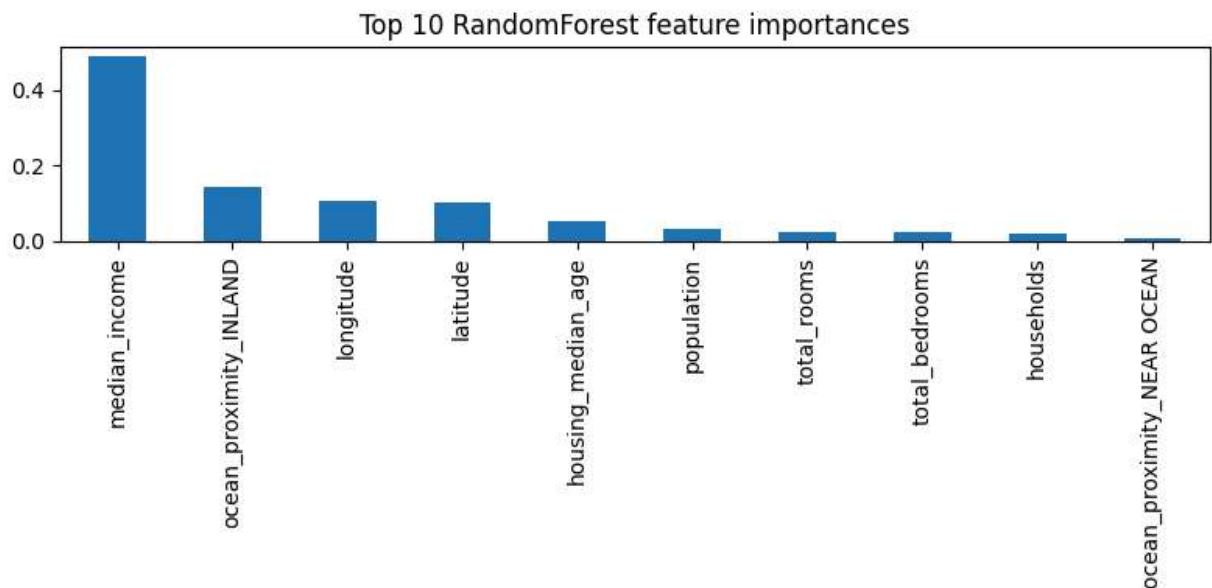


RandomForest - CV RMSE: 50664.3501 | Test RMSE: 50047.8027 | Test R²: 0.8089

Top 15 feature importances (RandomForest):

	0
median_income	0.488945
ocean_proximity_INLAND	0.141256
longitude	0.107519
latitude	0.102250
housing_median_age	0.052285
population	0.032530
total_rooms	0.023428
total_bedrooms	0.023092
households	0.017705
ocean_proximity_NEAR OCEAN	0.006470
ocean_proximity_<1H OCEAN	0.003334
ocean_proximity_NEAR BAY	0.000760
ocean_proximity_ISLAND	0.000427

dtype: float64



Step 10 - results summary

```
results = []
```

```
def eval_pipeline(name, pipeline):
    preds = pipeline.predict(X_test)
    return {
```

```

        'model': name,
        'test_rmse': np.sqrt(mean_squared_error(y_test, preds)),
        'test_r2': r2_score(y_test, preds)
    }

# ensure models are fitted
for name, pipe in [('LinearRegression', lr_pipeline),
                  ('Ridge', ridge_pipeline),
                  ('RandomForest', rf_pipeline)]:
    results.append(eval_pipeline(name, pipe))

results_df = pd.DataFrame(results).sort_values('test_rmse').reset_index(drop=True)
print("Model comparison (sorted by test RMSE):")
display(results_df)

```

Model comparison (sorted by test RMSE):

	model	test_rmse	test_r2	
0	RandomForest	50047.802676	0.808855	
1	LinearRegression	70384.406132	0.621953	
2	Ridge	70387.214640	0.621923	



Next steps:

[Generate code with results_df](#)

[New interactive sheet](#)

Step 11 - Save best model

```

sort joblib
st_idx = results_df['test_rmse'].idxmin()
st_model_name = results_df.loc[st_idx, 'model']

```